

Applicants Migration and Model

We are now going to start to implement job application submissions. Users can apply for jobs and submit their resumes. We will create a new table to store the applicants and migrate it to the database.

Create Applicants Table

Let's create a new table to store the applicants. We will create a new migration file for this.

```
php artisan make:migration create_applicants_table
```

Open the file `database/migrations/TIMESTAMP_create_applicants_table.php` and add the following code:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('applicants', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->foreignId('job_id')->constrained('job_listings')->onDelete('cascade');
            $table->string('full_name');
            $table->string('contact_phone')->nullable();
            $table->string('contact_email');
            $table->text('message')->nullable();
            $table->string('location')->nullable();
            $table->string('resume_path');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('applicants');
    }
}
```

```
}  
};
```

We have the following columns:

- `job_id`: The ID of the job that the application is for.
- `user_id`: The ID of the user who submitted the application.
- `full_name`: The full name of the applicant.
- `contact_phone`: The contact number of the applicant.
- `contact_email`: The email address of the applicant.
- `message`: A message from the applicant.
- `location`: The location of the applicant.
- `resume_path`: The path to the applicant's resume.
- `created_at`: The timestamp when the applicant was created.
- `updated_at`: The timestamp when the applicant was last updated.

Migrate the Applicants Table

Run the following command to migrate the applicants table to the database:

```
php artisan migrate
```

The `applicants` table will be created in the database.

Create Applicants Model

Let's create a new model for the applicants table. We will create a new model file for this.

```
php artisan make:model Applicant
```

Open the file `app/Models/Applicant.php` and add the following code:

```
<?php  
  
namespace App\Models;  
  
use Illuminate\Database\Eloquent\Factories\HasFactory;  
use Illuminate\Database\Eloquent\Relations\BelongsTo;  
use Illuminate\Database\Eloquent\Model;  
  
class Applicant extends Model  
{  
    use HasFactory;  
  
    protected $fillable = [  
        'job_id',
```

```
        'user_id',
        'full_name',
        'contact_phone',
        'contact_email',
        'message',
        'location',
        'resume_path',
    ];

    public function job(): BelongsTo
    {
        return $this->belongsTo(Job::class);
    }

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

Make sure that you import the `BelongsTo` class from the `Illuminate\Database\Eloquent\Relations\BelongsTo` namespace.

We have defined the fillable fields and the relationships with the `Job` and `User` models. An applicant belongs to a job and a user.

Job Model

Now we need to add the 1-to-many relationship between the `Job` and `Applicant` models. Open the file `app/Models/Job.php` and add the following method:

```
public function applicants(): HasMany
{
    return $this->hasMany(Applicant::class);
}
```

Import the `HasMany` class from the `Illuminate\Database\Eloquent\Relations\HasMany` namespace.

```
use Illuminate\Database\Eloquent\Relations\HasMany;
```

This method will return all the applicants for a given job.

User Model

Now we need to add the 1-to-many relationship between the `User` and `Applicant` models. It makes more sense to use the word `applications` as the relationship name. Open the file `app/Models/User.php` and add

the following method:

```
public function applications(): HasMany
{
    return $this->hasMany(Applicant::class, 'user_id');
}
```

Now that we have our table, model and relationships, let's work on the form in the next lesson.